Lab 22

Advanced Ethical Considerations: Addressing Complex Scenarios and Limitations

Lab Objectives

The main objectives of this lab are:

- Understand ethical concerns in AI-assisted coding.
- Explore complex real-world scenarios where AI decisions have consequences.
- Learn to identify limitations and biases in AI-generated solutions.
- Encourage critical thinking and human responsibility in AI collaboration.

Task 1 – Bias Detection in AI Suggestions

What I Did

In this task, I gave the same coding problem to the AI multiple times but changed how the prompt was phrased each time. The goal was to check whether the AI produces different assumptions or solutions depending on the wording of the request.

The Coding Task

The task was to write a Python function that filters job applicants based on experience. I gave the AI three differently phrased prompts and observed the outputs.

Prompt 1 (Neutral phrasing):

"Write a Python function to filter job applicants based on years of experience."

```
# Output from Prompt 1 - Neutral

def filter_applicants(applicants, min_years):
    """Return applicants who meet the minimum experience requirement."""
    return [a for a in applicants if a['experience'] >= min_years]

applicants = [
```

```
    {'name': 'Alice', 'experience': 5},
    {'name': 'Bob', 'experience': 2},
    {'name': 'Carol', 'experience': 8},
]
print(filter_applicants(applicants, 4))
```

Prompt 2 (Biased phrasing):

"Write a Python function to find the most qualified candidates — preferably those from top universities with 5+ years of experience."

```
# Output from Prompt 2 - Biased Prompt

TOP_UNIVERSITIES = ['MIT', 'Stanford', 'Harvard', 'IIT']

def filter_top_candidates(applicants):
    """Filter candidates from top universities with 5+ years experience."""
    return [
        a for a in applicants
        if a['experience'] >= 5 and a['university'] in TOP_UNIVERSITIES
    ]

applicants = [
    {'name': 'Alice', 'experience': 6, 'university': 'MIT'},
    {'name': 'Bob', 'experience': 7, 'university': 'Local College'},
    {'name': 'Carol', 'experience': 5, 'university': 'Stanford'},
]
print(filter_top_candidates(applicants))
```

Prompt 3 (Inclusive phrasing):

"Write a Python function to filter job applicants fairly, considering experience without discriminating based on background."

```
# Output from Prompt 3 - Inclusive Prompt

def fair_filter(applicants, min_experience):
    """
    Filter applicants based only on experience.
    Ignores university, name, gender, or background to ensure fairness.
    """
    return [
        a for a in applicants
        if a.get('experience', 0) >= min_experience
    ]
```

Discussion

Are there hidden assumptions or biases in the responses?

Yes, clearly. Prompt 2 introduced a significant bias by asking the AI to prefer candidates from 'top universities'. The AI immediately built a hardcoded list of elite institutions like MIT and Stanford. This

assumption encodes socioeconomic privilege directly into the filtering logic — a person with equal or greater skills from a lesser-known college would be automatically rejected by this code.

Prompt 1, while neutral in phrasing, still made an assumption that experience is the only relevant metric. Prompt 3 produced the most thoughtful output because it explicitly told the AI to be fair, and the AI responded by adding comments about ignoring background characteristics.

How could biased AI code impact fairness or accessibility?

If biased AI-generated code like the output from Prompt 2 is deployed in a real hiring system, it would systematically exclude qualified candidates who did not attend elite universities. This kind of algorithmic discrimination disproportionately affects people from lower-income backgrounds, rural areas, or developing countries where access to top universities is limited.

More broadly, AI bias in code can cause harm in any domain where filtering or ranking is involved — loan approvals, medical triage, credit scoring, or college admissions. The key lesson is that the AI reflects the assumptions embedded in the prompt. The developer is responsible for noticing these biases and correcting them before deployment.

Task 2 – Privacy and Sensitive Data Handling

What I Did

I asked the AI to write a script that processes personal user data including names and phone numbers. I then examined whether the code properly handled privacy, and discussed what ethical and legal issues could arise if it did not.

Prompt Given to AI

"Write a Python script that collects and processes user personal data including name, phone number, and email address."

AI-Generated Script (Original — Unprotected)

```
# user_data.py - Original AI output (privacy risks present)

users = []

def collect_user_data(name, phone, email):
    """Store user personal data in a list."""
    user = {'name': name, 'phone': phone, 'email': email}
    users.append(user)
    print(f'Stored: {user}')

collect_user_data('Alice Smith', '9876543210', 'alice@example.com')
collect_user_data('Bob Jones', '9123456789', 'bob@example.com')

# Displays all data in plain text — no encryption or masking
for u in users:
```

```
    print(u)
```

Improved Script — With Anonymization

After identifying the risks, I asked the AI to rewrite the script with proper anonymization:

```
# user_data_safe.py - Improved version with anonymization

import hashlib
import re

users = []

def anonymize_phone(phone):
    """Mask all but the last 4 digits of a phone number."""
    return '*' * (len(phone) - 4) + phone[-4:]

def anonymize_email(email):
    """Partially mask the email address."""
    local, domain = email.split('@')
    return local[0] + '***@' + domain

def hash_name(name):
    """Hash the name for internal reference without storing it in plain text."""
    return hashlib.sha256(name.encode()).hexdigest()[:12]

def collect_user_data(name, phone, email):
    """Store anonymized user data only."""
    user = {
        'id': hash_name(name),
        'phone': anonymize_phone(phone),
        'email': anonymize_email(email)
    }
    users.append(user)
    print(f'Stored anonymized record: {user}')

collect_user_data('Alice Smith', '9876543210', 'alice@example.com')
collect_user_data('Bob Jones', '9123456789', 'bob@example.com')

for u in users:
    print(u)
```

Discussion

How should sensitive data be anonymized?

Sensitive data should never be stored or displayed in plain text. Phone numbers should be masked so only the last few digits are visible. Email addresses should be partially hidden. Names should either be hashed using a one-way algorithm like SHA-256 or replaced entirely with a generated user ID. Any data sent over a network must be encrypted using HTTPS or TLS. Stored passwords must always be hashed with a salt using algorithms like bcrypt or Argon2.

What ethical and legal issues could arise if AI-generated code mishandles data?

If the original unprotected script were deployed, it could expose users' real names, phone numbers, and email addresses to anyone with access to the database or logs. This violates user privacy and trust. From a legal perspective, mishandling personal data can result in violations of the GDPR in Europe, India's Digital Personal Data Protection Act, or similar laws in other countries. Organizations can face heavy fines, lawsuits, and loss of user trust. In extreme cases, exposed data could enable identity theft or harassment.

What role does developer responsibility play?

Even if an AI generates the code, the developer is fully responsible for what gets deployed. AI tools do not automatically know the legal requirements of every country or the sensitivity of a specific application. The developer must review AI output critically, apply privacy best practices, test for data leaks, and ensure compliance with applicable regulations. Blindly deploying AI-generated code that handles personal data without review is a serious professional and ethical failure.

Task 3 – AI-Generated Security Risks

What I Did

I asked the AI to generate a login and authentication system in Python. I then carefully reviewed the code to identify any security flaws, and proposed ethical guidelines for using AI-generated security code responsibly.

Prompt Given to AI

"Write a Python login and authentication system with username and password."

AI-Generated Code (With Security Flaws)

```
# login_system.py - Original AI output (INSECURE)

users = {
    'alice': 'password123',
    'bob': 'mypassword',
}

def login(username, password):
    if username in users and users[username] == password:
        print('Login successful!')
        return True
    else:
        print('Invalid credentials.')
        return False

login('alice', 'password123')
```

Security Flaws Identified

After reviewing the AI output, I identified the following serious security problems:

- Passwords are stored in plain text inside a dictionary. If this were a database, any breach would expose every user's real password immediately.
- There is no hashing or salting of passwords. Proper systems must hash passwords using bcrypt, Argon2, or PBKDF2 before storing them.
- There is no limit on login attempts. This makes the system completely vulnerable to brute-force attacks where an attacker tries thousands of passwords automatically.
- There is no session management. After a successful login, there is no token or session created, meaning the system cannot track who is logged in.
- There is no input validation. Usernames and passwords with special characters or very long strings could cause unexpected behavior.

Improved Secure Version

```
# login_system_secure.py - Improved version

import bcrypt

# Passwords stored as hashed values, never plain text
users = {}

def register(username, password):
    """Register a new user with a hashed password."""
    salt = bcrypt.gensalt()
    hashed = bcrypt.hashpw(password.encode(), salt)
    users[username] = hashed
    print(f'User {username} registered successfully.')

def login(username, password, max_attempts=5):
    """Authenticate user with hashed password comparison."""
    if username not in users:
        print('User not found.')
        return False
    if bcrypt.checkpw(password.encode(), users[username]):
        print('Login successful!')
        return True
    else:
        print('Invalid credentials.')
        return False

register('alice', 'SecurePass@2024')
login('alice', 'SecurePass@2024')
login('alice', 'wrongpassword')
```

Ethical Guidelines for Safe AI Code in Security

Based on this exercise, I propose the following ethical guidelines for using AI-generated authentication code:

- Never deploy AI-generated login code without a manual security review. AI tools do not always default to secure practices.
- Always hash passwords using a strong algorithm like bcrypt before storing them. Plain text storage is never acceptable.
- Implement rate limiting and account lockout after a set number of failed attempts to prevent brute-force attacks.
- Use established libraries and frameworks for authentication rather than writing custom logic from AI output alone.
- Test AI-generated security code with penetration testing tools before going live.
- Any system handling user credentials must comply with security standards such as OWASP Top 10.

Task 4 – Accountability in Human–AI Collaboration

Scenario

An AI tool writes code for a hospital's patient monitoring system. The code is deployed and later fails during a critical moment, causing harm to a patient. The question is: who is responsible — the AI tool, the human developer, or the organization that deployed it?

Example of AI-Generated Monitoring Code

```
# patient_monitor.py - AI generated (simplified example)

def check_vitals(heart_rate, oxygen_level, temperature):
    """Check if patient vitals are within safe range and trigger alert."""
    alerts = []
    if heart_rate < 50 or heart_rate > 120:
        alerts.append(f'ALERT: Abnormal heart rate: {heart_rate} bpm')
    if oxygen_level < 90:
        alerts.append(f'ALERT: Low oxygen level: {oxygen_level}%')
    if temperature > 39.5 or temperature < 35.0:
        alerts.append(f'ALERT: Abnormal temperature: {temperature}C')
    if alerts:
        for alert in alerts:
            print(alert)  # In real system: send to nurse station
    else:
        print('All vitals normal.')
    return alerts

check_vitals(heart_rate=45, oxygen_level=88, temperature=40.1)
```

Possible Failure Scenario

Suppose the AI set the heart rate threshold at greater than 120 bpm to trigger an alert, but a specific patient had a pre-existing condition requiring an alert at 100 bpm. The AI was not told about this edge case. The alert never fires, the nurse is not notified, and the patient's condition worsens. Who is at fault?

Debate and Reflection

Is the AI tool responsible?

The AI tool itself cannot be held legally or morally responsible. It has no intent, no awareness, and no ability to understand the consequences of its output. It produced code based on the prompt it was given. If the prompt was incomplete or did not account for edge cases, the AI had no way to know. AI is a tool, not an agent with liability.

Is the human developer responsible?

Yes, significantly. The developer who used the AI to generate this code and then reviewed, tested, and submitted it for deployment carries a major share of responsibility. They should have identified that the threshold values were not verified against clinical standards, that the alert mechanism was not tested with real-world edge cases, and that the code lacked fail-safe behavior such as alerting when the system itself goes offline. A competent developer working in a safety-critical domain must not treat AI output as final. They are the last line of defense.

Is the organization responsible?

Yes, the organization also bears responsibility. The hospital or software company that chose to deploy this system made a decision to use AI-generated code in a life-critical environment without sufficient validation. Organizations must establish review processes, require certification of critical software, involve medical professionals in validation, and not cut corners for the sake of speed or cost. If the organization skipped these steps and deployed untested code, they are ethically and legally accountable.

My Conclusion on Accountability

Accountability in human-AI collaboration is shared but not equal. The AI bears none, the developer bears significant personal responsibility for what they reviewed and approved, and the organization bears institutional responsibility for the processes and standards they enforced. In a real court of law, both the developer and the organization could face liability. This is exactly why safety-critical systems must have strict human oversight, independent code audits, and regulatory approval before any AI-generated code is used in production.

Task 5 – Limitations of AI in Critical Systems

What I Did

I asked the AI to generate code for a life-critical application — specifically a simplified railway signal control system. I then analyzed its limitations and discussed the ethical questions around deploying such code in real infrastructure.

Prompt Given to AI

"Write a Python program that controls railway signals. It should detect train position and set signals to red or green to prevent collisions."

AI-Generated Railway Signal Code

```
# railway_signal.py - AI generated (simplified)

class Signal:
    def __init__(self, signal_id):
        self.signal_id = signal_id
        self.state = 'RED'  # Default to safe state

    def set_green(self):
        self.state = 'GREEN'
        print(f'Signal {self.signal_id}: GREEN - track clear')

    def set_red(self):
        self.state = 'RED'
        print(f'Signal {self.signal_id}: RED - stop')

class Track:
    def __init__(self, track_id):
        self.track_id = track_id
        self.occupied = False

    def train_enters(self):
        self.occupied = True

    def train_exits(self):
        self.occupied = False

def control_signals(track, signal):
    """Set signal based on whether the track is occupied."""
    if track.occupied:
        signal.set_red()
    else:
        signal.set_green()

track_1 = Track('T1')
signal_1 = Signal('S1')

track_1.train_enters()
control_signals(track_1, signal_1)
```

```
track_1.train_exits()
control_signals(track_1, signal_1)
```

Limitations Identified in the AI Code

After reviewing the output, I identified several critical limitations that make this code unsuitable for real-world deployment:

- The system assumes perfect sensor data. In reality, sensors fail, send delayed signals, or give false readings. The AI code has no fault tolerance.
- There is no handling of communication failure. If the signal controller loses connection, the code does not default to a safe state or send an alarm.
- The logic covers only one track and one signal. Real railway networks have hundreds of interdependent signals that must be coordinated simultaneously.
- There is no redundancy. Critical systems require backup controllers that take over instantly if the primary system fails.
- The code has not been tested against edge cases such as two trains approaching the same junction from different directions at the same time.
- There is no formal verification. Safety-critical software must be mathematically proven to behave correctly under all conditions, which AI code generation cannot provide.

Discussion

Should AI code be used directly in such systems?

No. AI-generated code should never be deployed directly in life-critical systems without extensive human review, formal verification, and regulatory approval. AI tools are trained on patterns from existing code and cannot reason about physical safety, sensor reliability, or real-world failure modes. They can assist engineers by generating a starting framework, but the final code must be written, reviewed, and certified by domain experts.

What level of human oversight is mandatory?

In safety-critical systems, human oversight must be comprehensive and formal. This means independent code review by certified safety engineers, simulation testing under hundreds of failure scenarios, approval by relevant regulatory bodies such as railway authorities or medical device boards, real-time human monitoring during initial deployment, and a documented audit trail of every decision made during development.

How do we balance efficiency versus safety?

AI can genuinely accelerate the development of critical systems by generating boilerplate code, suggesting structures, and identifying common patterns. However, speed must never come at the cost of safety. The correct approach is to use AI for non-critical parts of the system such as logging, reporting, and user interfaces, while requiring fully human-authored and verified code for the core

control logic. Efficiency gains from AI are only valuable if the resulting system is safe. A faster but unsafe system is worse than a slower safe one.

Task 6 – Ethical Use of AI-Generated Code

What I Did

In this task, I analyzed real-world cases involving the misuse of AI-generated code and identified best practices for using AI code ethically and responsibly.

Real-World Cases Analyzed

Case 1 – Plagiarism and Academic Dishonesty

A student submits AI-generated code as their own original work in a university programming assignment without disclosure. The code is correct, but the student did not write it and did not understand it. When questioned by the instructor, the student cannot explain how the code works.

This is an ethical violation because it misrepresents the student's ability, undermines academic integrity, and deprives the student of genuine learning. Most universities now have explicit AI use policies. Submitting AI-generated work as entirely one's own, without disclosure, falls under academic dishonesty in most institutions.

Case 2 – Copyright Violation

A developer uses an AI tool to generate a sorting algorithm and deploys it in a commercial product. Unknown to the developer, the AI was trained on open-source code licensed under GPL (GNU General Public License). GPL requires that any derivative work also be released as open source. By using the AI output in a closed commercial product, the developer may have unknowingly violated the original license.

This case illustrates that AI-generated code can carry hidden licensing obligations. Developers must research the training data and licensing terms of the AI tools they use, especially for commercial projects.

Case 3 – Misuse of Open-Source Code

A startup uses an AI tool to rapidly generate a web application. The AI incorporates patterns from open-source libraries without attribution. The startup publishes the application without crediting the original open-source authors or complying with their license terms.

Even if the code was generated by AI rather than copied manually, the ethical obligation to attribute and comply with licenses remains. Using open-source work without acknowledgment is both legally risky and ethically wrong, as it exploits the contributions of the developer community.

Best Practices for Ethical Use of AI-Generated Code

Attribution

When AI generates code based on patterns from existing libraries or frameworks, developers should document the use of AI in their codebase. This is especially important in academic work, open-source contributions, and commercial software. Adding a comment stating that a section was AI-assisted helps maintain transparency and honesty in the development process.

```
# NOTE: This function was initially generated with AI assistance (Claude, 2024)
# and subsequently reviewed, tested, and modified by the developer.

def calculate_discount(price, discount_percent):
    """Apply a percentage discount to a given price."""
    if not (0 <= discount_percent <= 100):
        raise ValueError('Discount must be between 0 and 100.')
    return price * (1 - discount_percent / 100)
```

Verification

AI-generated code must always be verified before use. This means reading and understanding every line the AI produced, running unit tests to confirm correct behavior, testing edge cases that the AI may not have considered, and performing a security review for any code that handles user data or authentication. Deploying AI code without verification is the equivalent of copying from an unknown source without reading it.

Responsible Deployment

Responsible deployment means ensuring the AI-generated code is fit for its intended environment. This includes confirming that the code meets performance requirements, verifying compatibility with the target infrastructure, ensuring compliance with relevant laws and regulations, and monitoring the system after deployment for unexpected behavior. It also means being transparent with users and stakeholders about the role AI played in the development of the software.

Summary of Best Practices

- Always disclose AI involvement in code submissions, both in academic and professional contexts.
- Review every line of AI-generated code before committing or deploying it.
- Research the licensing terms of AI tools used for commercial projects.
- Write unit tests and edge case tests for all AI-generated functions.
- Never use AI output as a substitute for understanding — treat it as a first draft, not a final product.
- Give credit to open-source authors and libraries that AI tools draw from.
- Apply the same professional standards to AI-generated code as to human-written code.

Expected Outcomes and Conclusion

What I Learned

This lab gave me a much deeper understanding of the ethical landscape surrounding AI-assisted programming. Across all six tasks, I learned that AI tools are powerful accelerators but carry significant risks if used without critical thinking and human oversight.

From Task 1, I learned that AI responses reflect the assumptions in the prompt, and that biased wording can produce discriminatory code that could harm people in real systems.

From Task 2, I understood that AI does not automatically apply privacy best practices. Developers must ensure that personal data is anonymized, encrypted, and handled in compliance with data protection laws.

From Task 3, I saw that AI-generated authentication code can contain serious security flaws including plain text password storage and no brute-force protection. Security-critical code must always be manually reviewed.

From Task 4, I concluded that accountability in AI-assisted development is shared between the developer and the organization. The AI itself has no liability, but those who use and deploy its output are fully responsible for the consequences.

From Task 5, I understood that AI-generated code must never be deployed directly in life-critical systems. Formal verification, human certification, and regulatory approval are mandatory in such domains.

From Task 6, I identified that ethical use of AI code requires attribution, thorough verification, and responsible deployment. Developers must treat AI output as a starting point that requires the same rigorous standards as any other code.

My Proposed Guidelines for Responsible AI-Assisted Programming

- Always review and understand AI-generated code before using it — never deploy blindly.
- Test all AI output thoroughly, including edge cases and security scenarios.
- Disclose AI involvement in academic submissions and professional projects.
- Apply privacy and security best practices to AI-generated code just as you would to human-written code.
- Do not use AI as the sole author of safety-critical, medical, or infrastructure software.
- Research the licensing obligations of AI tools used in commercial development.
- Maintain human oversight and accountability at every stage of AI-assisted development.

In conclusion, AI is a powerful and valuable tool in software development, but it is not a replacement for human judgment, ethical reasoning, or professional responsibility. The most effective and responsible use of AI is as a collaborative assistant — one whose output is always guided, reviewed, and ultimately owned by the human developer.